Computer Vision 1

Reflectance & Photometric Stereo
Image Processing Basics

Lecturer: Yen-Chia Hsu

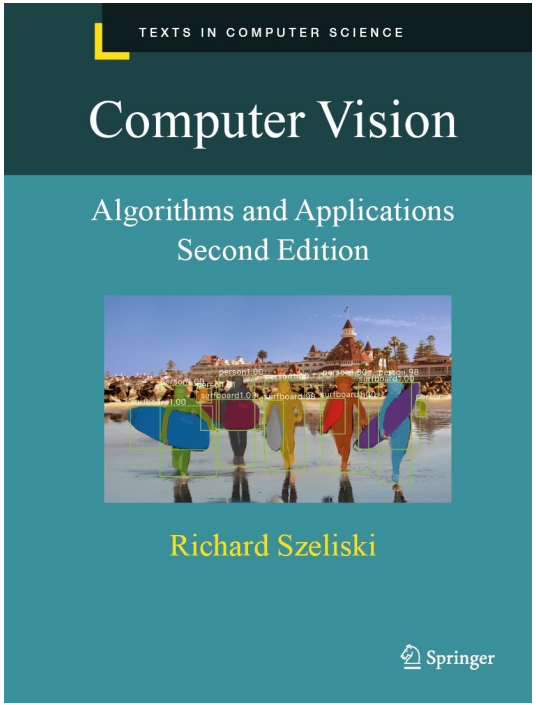
Sep 2026

Credit: This slide is inherited from Dr. Martin Oswald and Dr. Dimitris Tzionas

University of Amsterdam

Textbook

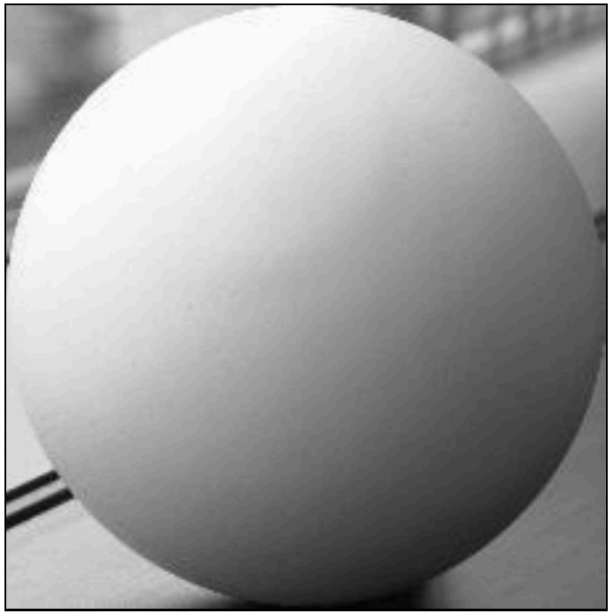
- 3.1.1
- 3.1.2
- 3.1.4
- 2.2.2
- 13.1.1

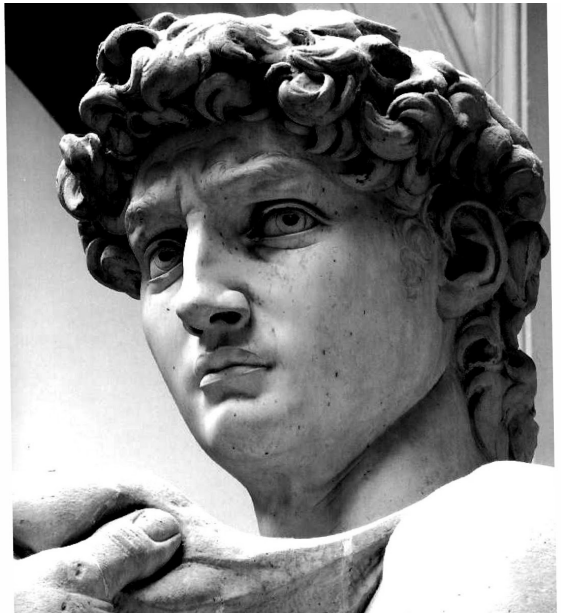


Outline

- Reflection and Photometric Stero
  - Reflection Models
  - Shape from Shading
  - Photometric Stereo
  - Intrinsic Decomposition
  - Applications
- Image Processessing Basics
  - Histogram
  - Histogram Modification
  - Histogram Equalization

Can we recover shape from shading?



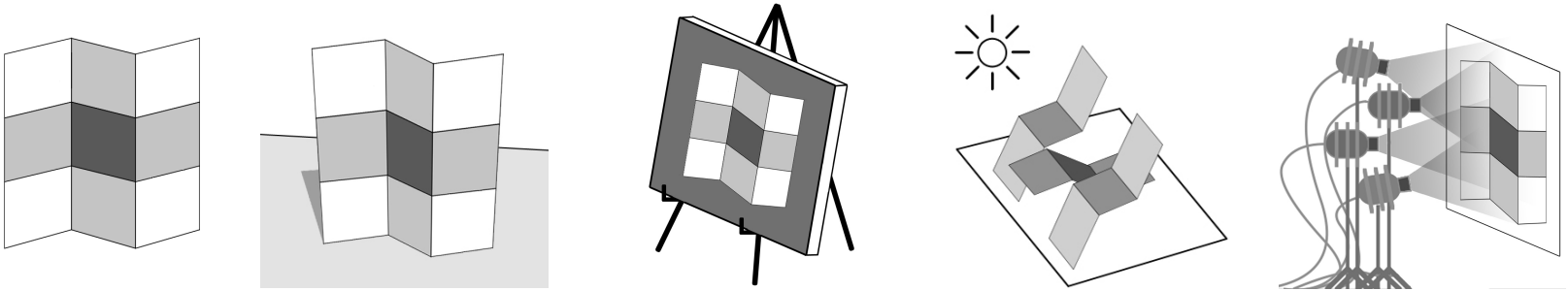




What is the relation between **intensity** and **shape**? Can we recover shape?

[Slide credit: Andreas Geiger]

Adelson and Pentland's Workshop Metaphor



(a) an image (b) a likely explanation (c) painter's explanation (d) sculptor's explanation (e) gaffer's explanation

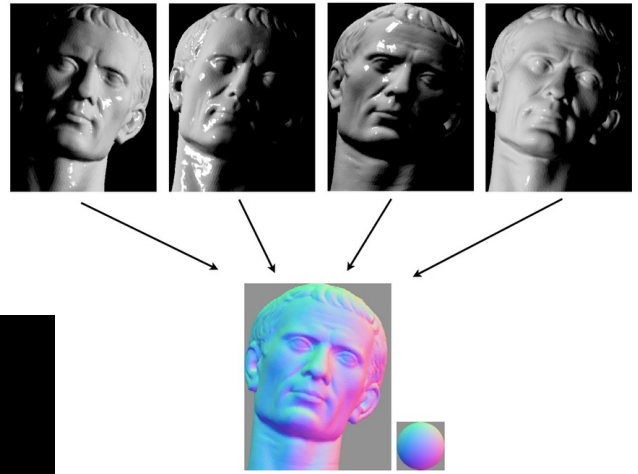
Can we recover **3D from a single image?**

- Inverting the graphics problem is very challenging
- The space of shapes, paint, and light that exactly reproduce an image is vast
- The image in (a) very likely corresponds to (b), but it could be a painting (c), sculpture (d), or an arrangement of lights (e)
- We are looking for a **simple explanation** and must use **prior knowledge**

[Slide credit: Andreas Geiger]

Motivation: Photometric Stereo

- Photometric stereo captures **multiple** images from the **same viewpoint**, but with **different light settings**
- Per-pixel estimation of **normal** and **albedo** or **material**
- Assumes far cameras/lights



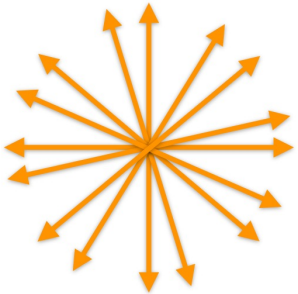
Surface normal, color coded



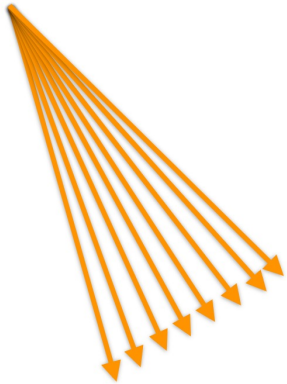
[Esper light stage]

Albedo is the measure of how much light is reflected from a surface, ranging from 0 (absorbing all lights) to 1 (reflecting all lights).

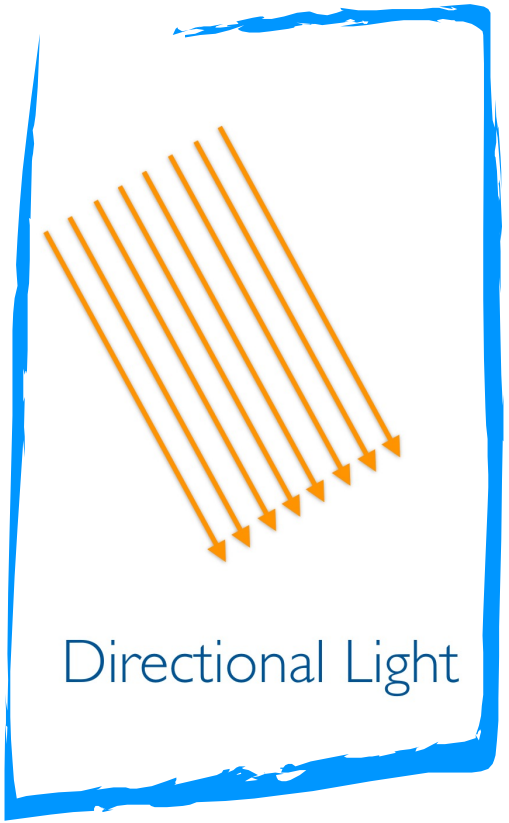
Light sources



Point Light



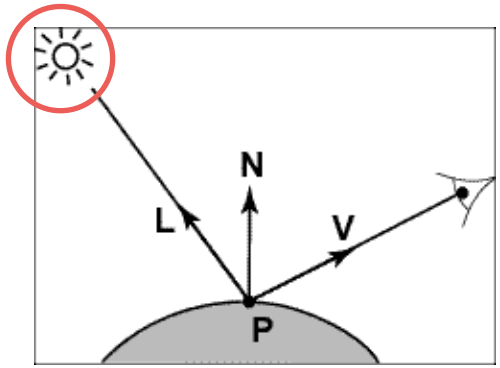
Spot Light



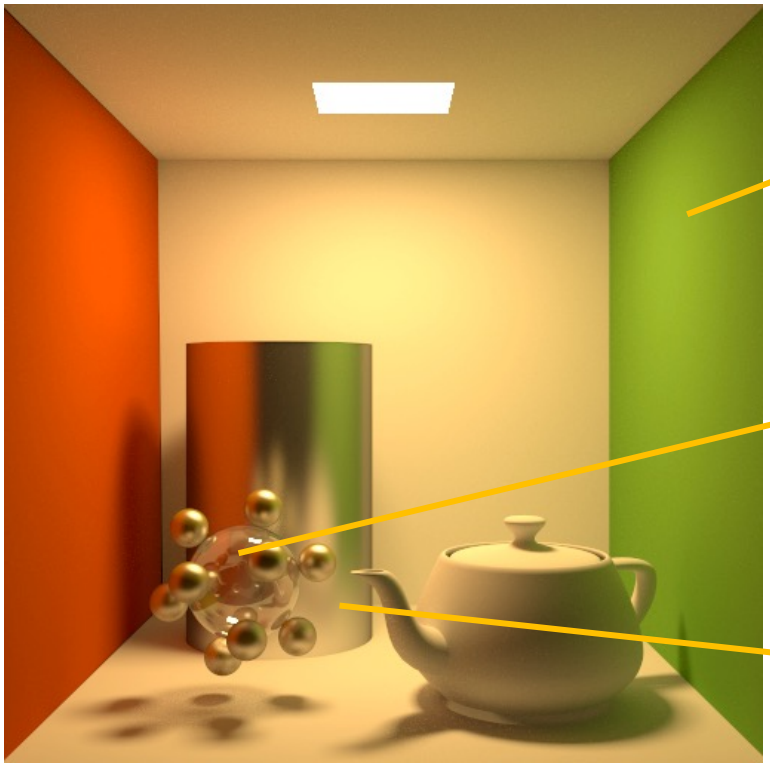
Directional Light

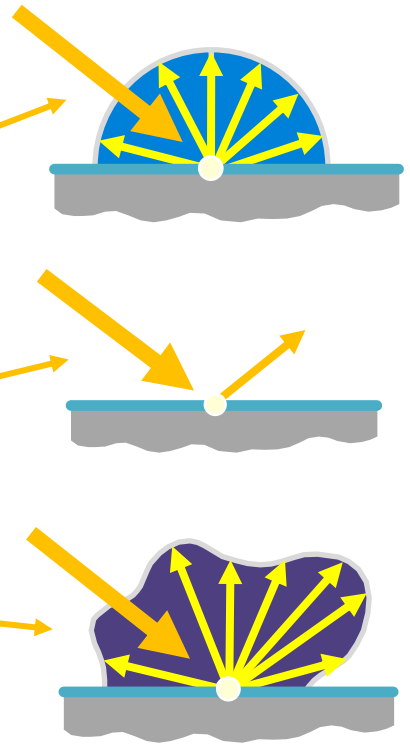
Directional Lighting and Reflectance

- Key property: all rays are parallel
- Equivalent to an infinitely distant point source



Materials - Three Forms





Ideal diffuse (Lambertian)

Light is reflected equally in all directions

Ideal specular

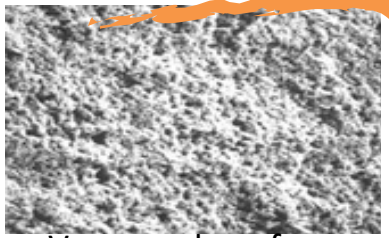
Light is reflected exactly like a mirror in only one direction

Directional diffuse

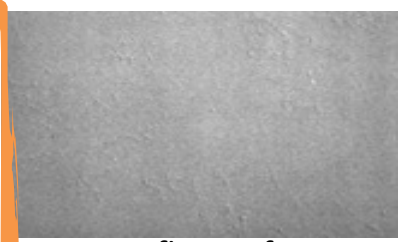
Light is reflected in different ways

[Slide credit: Kavita Bala]

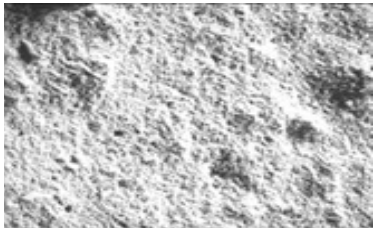
Reflectance - Three Forms



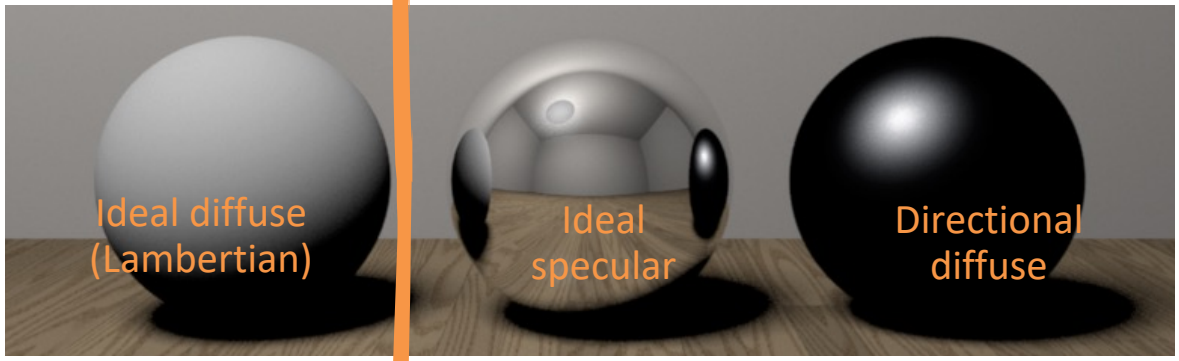
Very rough surface

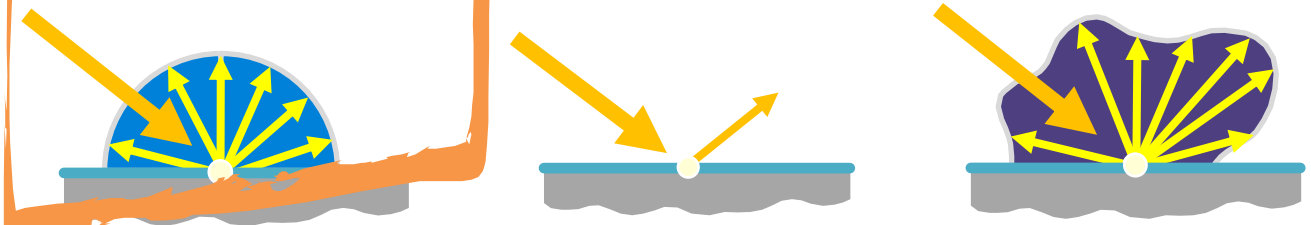


Very flat surface



Somewhere in between

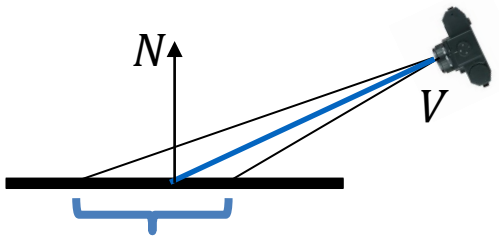




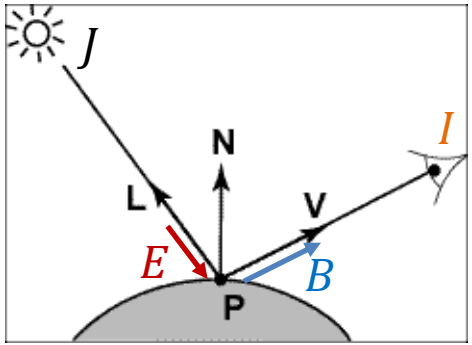
[Slide credit: Kavita Bala]

Terminology

- Light source intensity J: energy of the light source
- Irradiance E: incoming light energy (photons) that hit the surface
- Reflected energy B: light energy that goes out from the surface in a direction
- Radiance I: light energy that the camera sees (which is then converted to pixel values by the sensor)
- Light direction L: the direction of the incoming light
- Surface normal N: a vector that points out perpendicularly from a point on a surface
- View direction V: the direction of the viewer (e.g., camera)

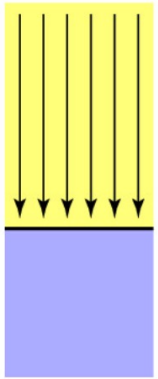


A pixel's radiance I is a collection of the reflected energy B emitted by an area that the pixel looks at.

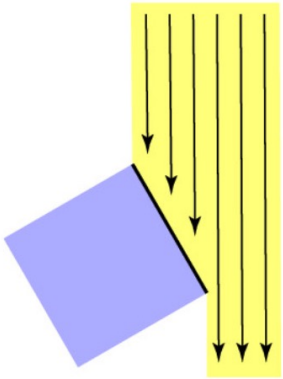


Lambert's Cosine Law for incoming light

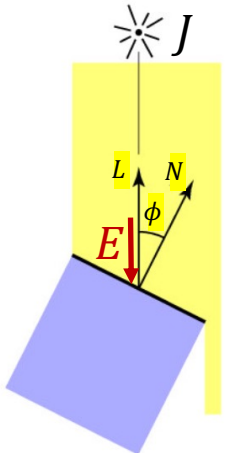
For the incoming light: Irradiance E at a unit surface is proportional to cosine of angle between surface normal N and light direction L.



Top face of cube receives a certain amount of power



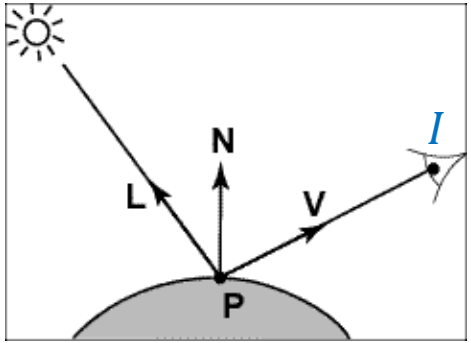
Top face of 60° rotated cube receives half power

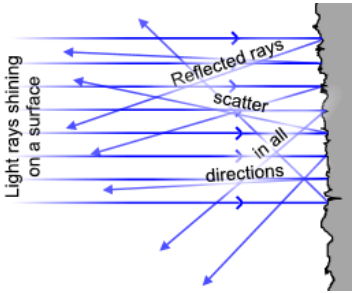


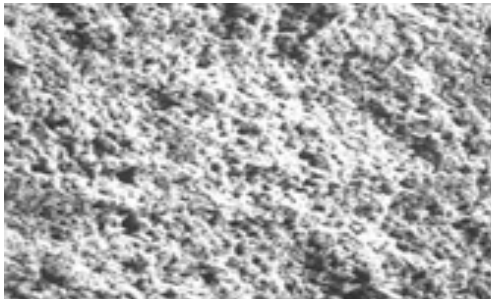
$$E \propto J \cos\phi$$
$$\propto J(N \cdot L)$$

Source -- https://cs184.eecs.berkeley.edu/sp24/lecture/11-27/radiometry-and-photometry

Reflectance - Ideal Diffuse





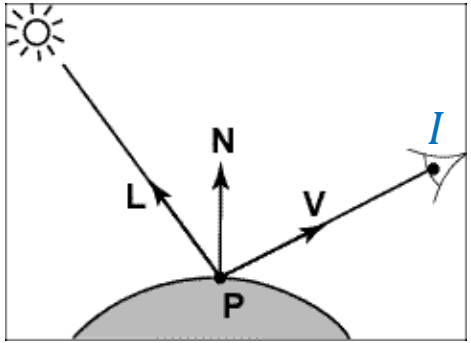


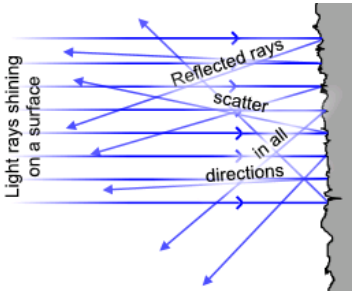
1. Using Lambert's Cosine Law, we can derive: Radiance I is proportional to cosine of angle between N (surface normal) and L (incoming light direction).

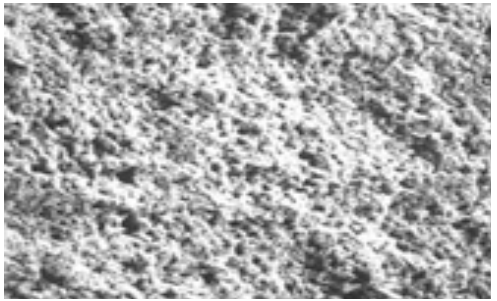
$$I \propto N \cdot L \propto \cos(\text{angle between } N \text{ and } L)$$

2. Measured image intensity is viewpoint-independent **(outgoing light)**

Reflectance - Ideal Diffuse







(dot product)

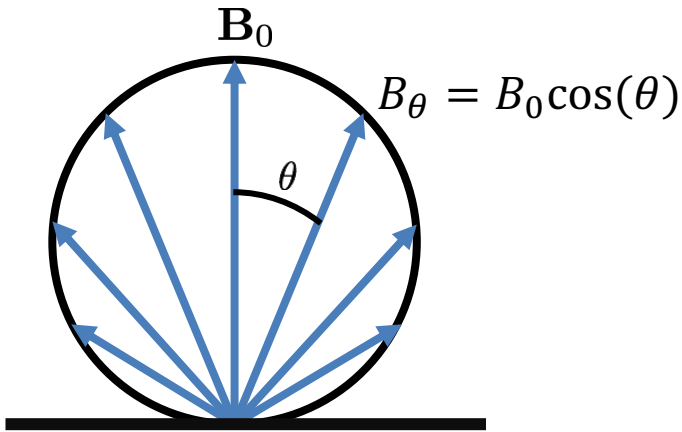
$$I \propto N \cdot L$$

Image intensity (scalar) $\propto$ Surface normal (vector) $\cdot$ Light direction (vector)

Image intensity $\propto$ cos(angle between N and L)

Lambert's Cosine Law for outgoing light

- Reflected energy B_θ to a given angle θ is proportional to $\cos(\theta)$

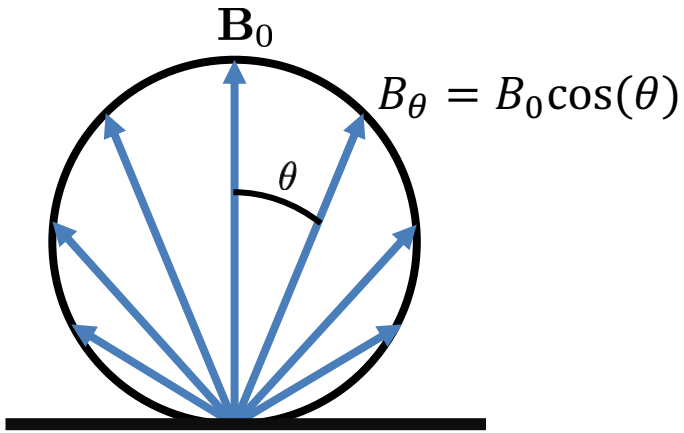


Lambert's cosine law: $B_\theta = B_0 \cos(\theta)$

Less light is reflected for larger angles

Lambertian appearance is view-independent

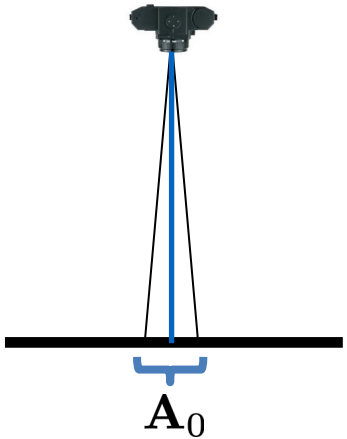
- Reflected energy B_θ to a given angle θ is proportional to $\cos(\theta)$



Lambert's cosine law: $B_\theta = B_0 \cos(\theta)$

Less light is reflected for larger angles

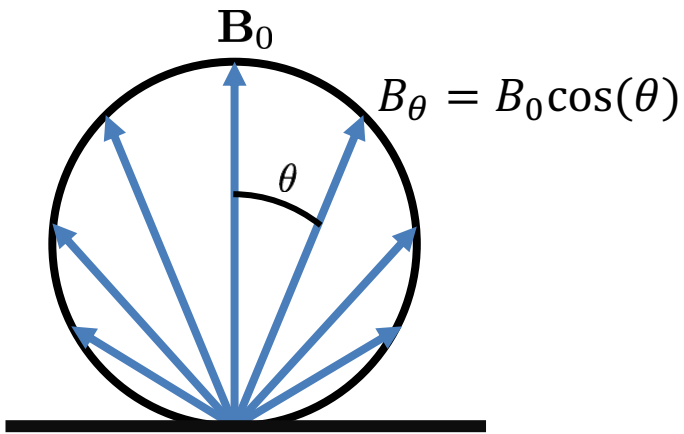
- But appearance is the same from every angle due to larger pixel footprint A_θ at larger angle θ



If camera points straight down, a single pixel sees the area A_0

Lambertian appearance is view-independent

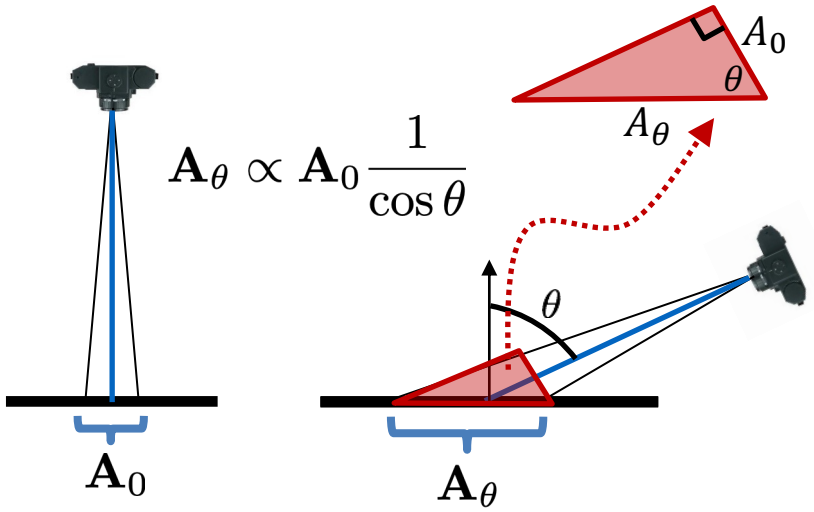
- Reflected energy B_θ to a given angle θ is proportional to $\cos(\theta)$



Lambert's cosine law: $B_\theta = B_0 \cos(\theta)$

Less light is reflected for larger angles

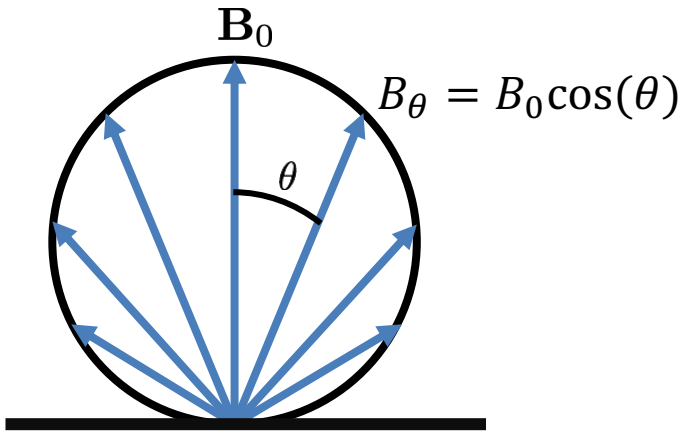
- But appearance is the same from every angle due to larger pixel footprint A_θ at larger angle θ



If camera tilts, the area that a single pixel sees stretches out to A_θ (this is geometry)

Lambertian appearance is view-independent

- Reflected energy B_θ to a given angle θ is proportional to $\cos(\theta)$

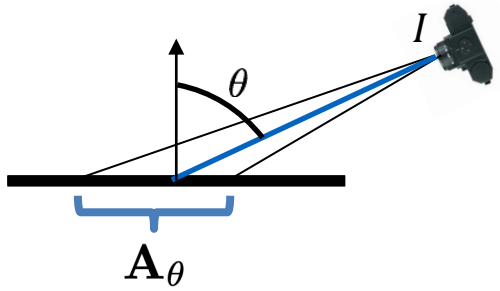


Lambert's cosine law: $B_\theta = B_0 \cos(\theta)$

Less light is reflected for larger angles

- But appearance is the same from every angle due to larger pixel footprint A_θ at larger angle θ

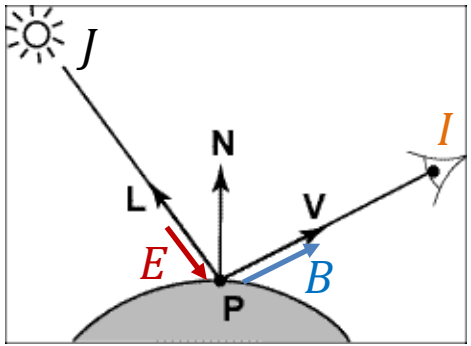
$$\mathbf{A}_\theta \propto \mathbf{A}_0 \frac{1}{\cos\theta}$$



Radiance I is proportional to light reflected toward the eye (B_θ) multiplied by the area (A_θ)

$$I \propto \mathbf{B}_0 \mathbf{A}_0 \cancel{\cos(\theta)} \frac{1}{\cancel{\cos(\theta)}}$$

Final Lambertian Image Formation Model



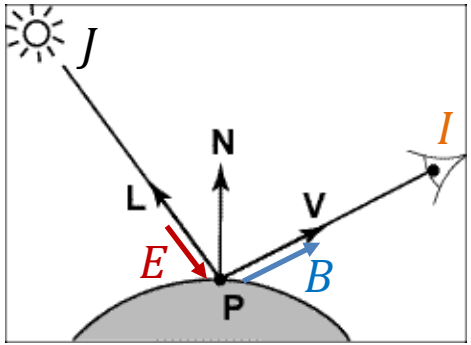
- We now have $I \propto B_0 A_0$
- We want to model the light absorption, so we define $B_0 \propto \rho E$, where ρ is the albedo
- According to the Lambert's cosine law for the incoming light, we have $E \propto J(N \cdot L)$
- Combine them all together, we have

$$I \propto \rho J(N \cdot L) A_0$$

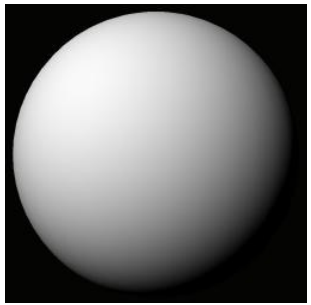
- There are other terms in the equation, such as camera gain. To simplify the math, we encode J into the magnitude of L and absorb all other parameters into k_d to get

$$I = k_d N \cdot L$$

Final Lambertian Image Formation Model

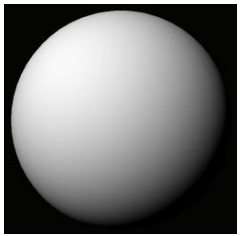


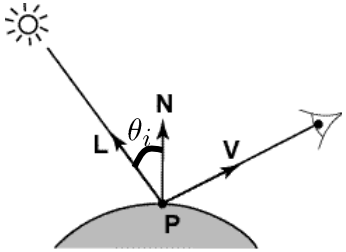
$$I = k_d N \cdot L$$



- We assume that the pixel value produced by the sensor is linearly proportional to the radiance (e.g., we get I from the RAW file format).
- Otherwise (e.g., brightness is compressed or adjusted by the camera), we need to calibrate the pixel values using the camera response function to reversely get the original raw image data.

A Single Image: Shape from Shading





Suppose (for now) $k_d = 1$

$$I = k_d \mathbf{N} \cdot \mathbf{L}$$
$$= \mathbf{N} \cdot \mathbf{L}$$
$$= cos\ \theta_i$$

The shape from shading task is to estimate the normals N given that other variables are known

You can directly measure angle between surface normal N and light source L

- Not enough information to compute surface shape (θ can be in any direction)
- But can be enough if you add some additional info, for example
  - assume a few of the normals are known (e.g., along silhouette: edge of objects or people)
  - constraints on neighboring normals — integrability and smoothness (surfaces are continuous)
  - For details, check https://www.youtube.com/watch?v=SLX7gPNHxSc
- Hard to get it to work well in practice
  - real objects rarely have constant albedo
  - but deep learning can help (using datasets of shapes and shadows)

Lightsource Position Ambiguity

Plates and bowls or upside down or not?

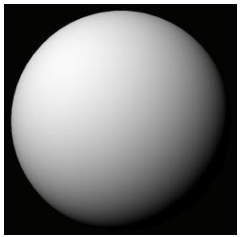


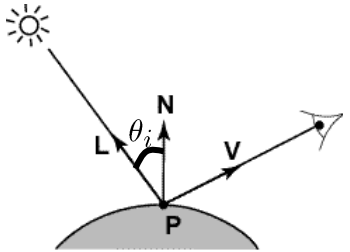
[https://www.good.is/optical-illusion-plates-and-bowls-upside-down-or-not]

Shape from Shading

Shape from Shading assumptions:

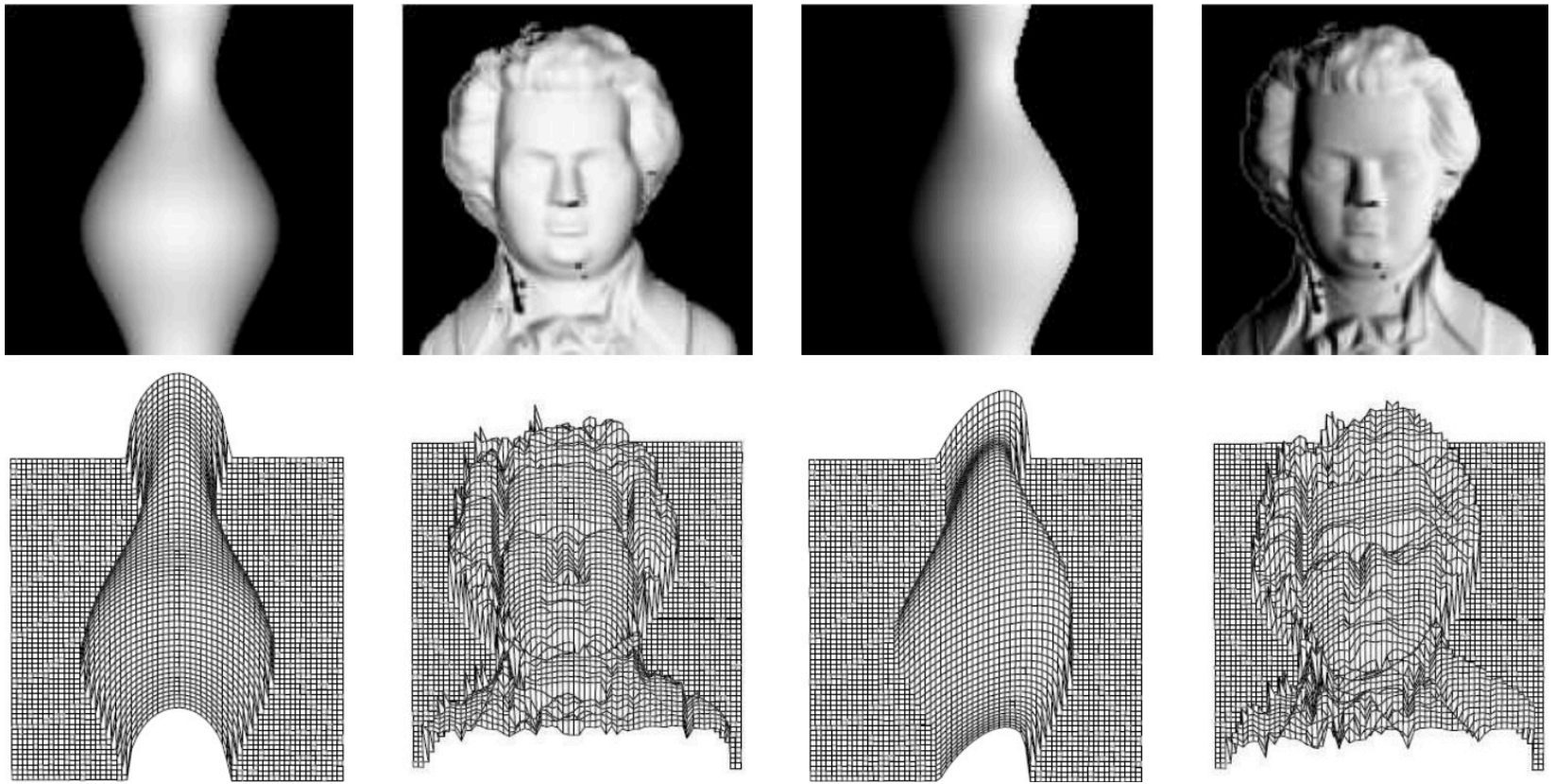
- Diffuse material with spatially constant albedo
  - Reduces the number of material parameters to 1 (which means assuming only one K_d and not multiple ones for different pixels)
- Known point light source at infinity
  - Keeps light direction L constant across all pixels
  - Makes L independent of object geometry and depth
- Known camera at infinity (orthography, no perspective distortion)
  - Keeps view direction V constant across all pixels
  - Makes V independent of object geometry and depth





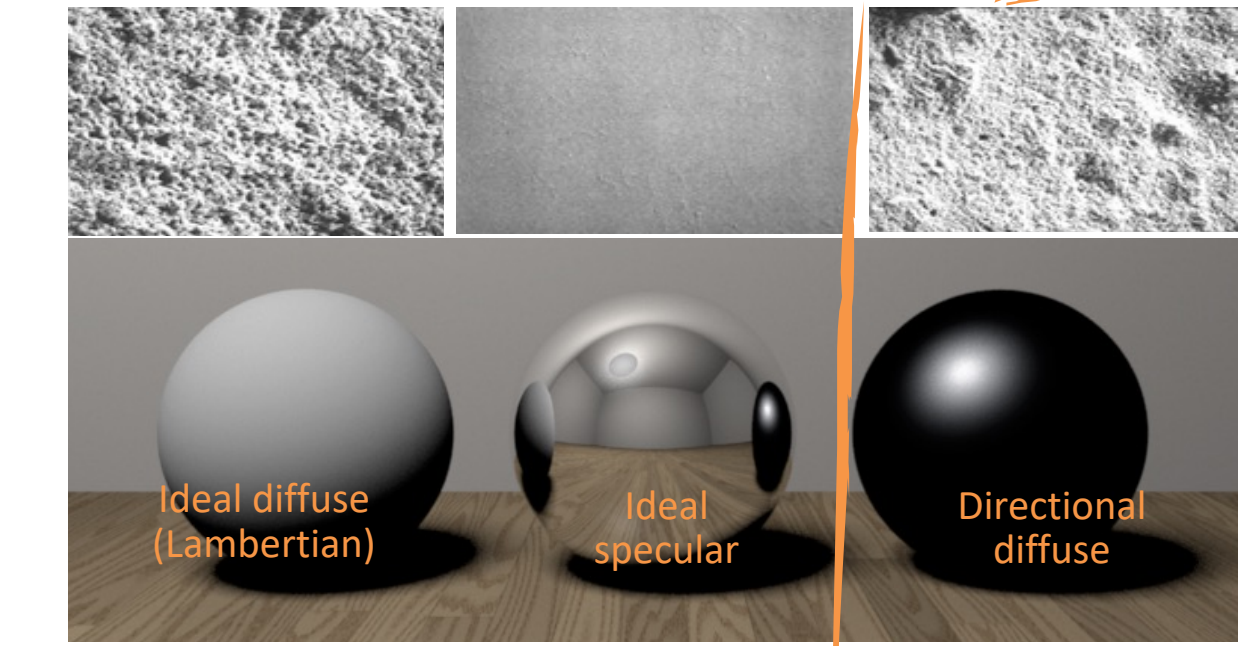
[Slide credit: Andreas Geiger]

Shape from Shading Results



[Slide credit: Andreas Geiger] [Frankot and Chellappa: A Method for enforcing integrability in shape from shading algorithms. TPAMI, 1988]

Reflectance - Three Forms



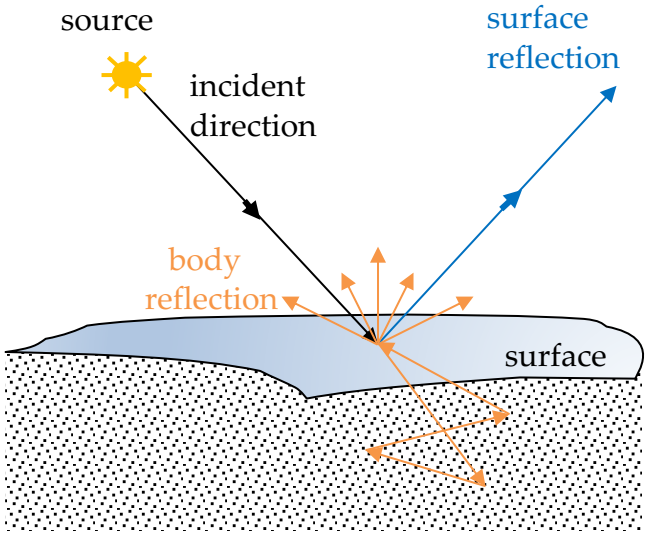
[Slide credit: Kavita Bala]

Mechanisms of Reflection

Body Reflection:

- Diffuse Reflection (we discussed this before)
- Matte Appearance
- Non-Homogeneous Medium (materials having composite of different elements mixed, such as wood fibers and air pockets for papers)
- Clay, paper, etc.





Many materials exhibit both Reflections:





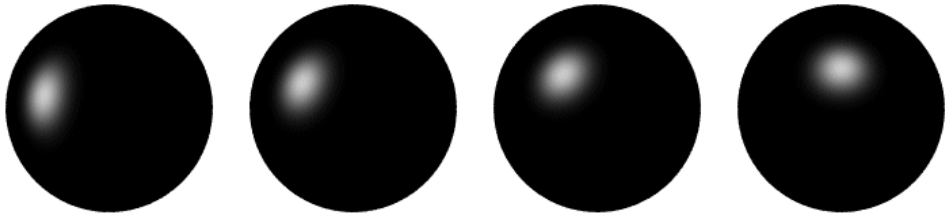
Surface Reflection:

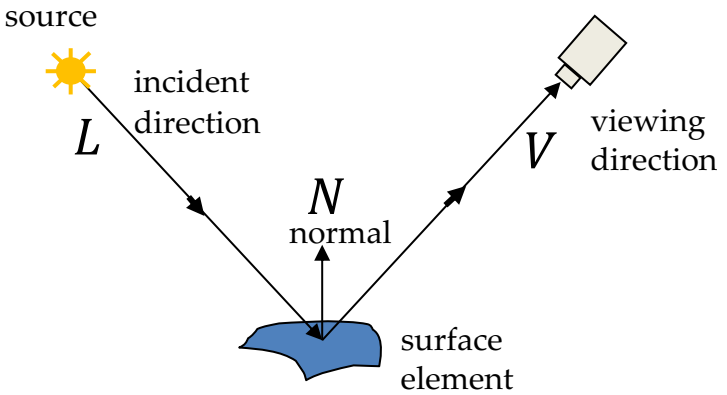
- Specular Reflection (like a mirror)
- Glossy Appearance
- Bright and shinny "Highlights"
- Dominant for Metals



Computing Light Source Directions

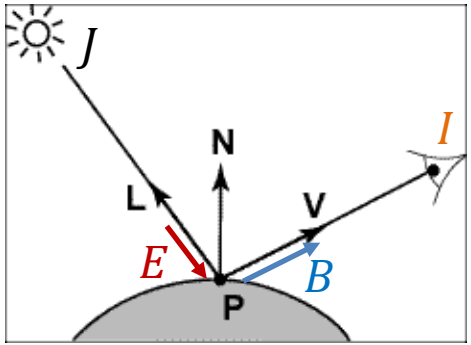
- Trick: Place a mirror ball (perfectly specular) in the scene. We know the surface normal N for every pixel on the surface.



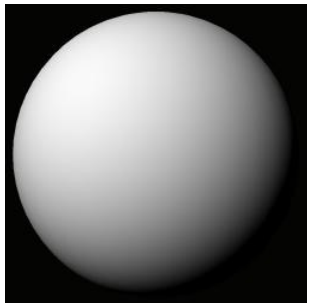


- The location of the highlight is determined by the light source direction. We find the brightest white pixel, which is the spot that the light is reflected perfectly into the camera.
- We also know the camera view angle V. Then, according to physics (angle of incidence equals angle of reflection), we can reversely get the light direction L.

Problem in our previous simple model



$$I = k_d N \cdot L$$



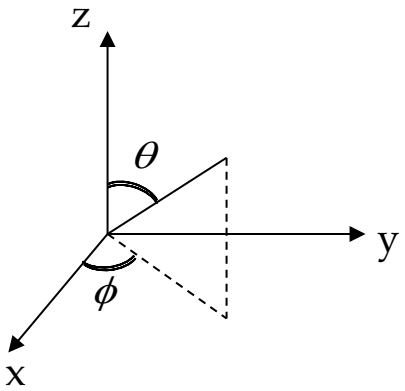
- For real-world materials, the reflectiveness changes depending on the viewing angle, which is not considered in the Lambertian image formation model (view-independent).
- We can do something better than this by replacing k_d with the BRDF (Bidirectional Reflectance Distribution Function), which becomes a view-dependent model.

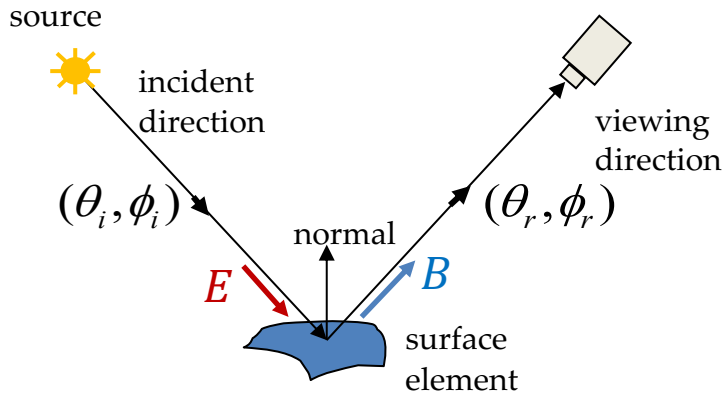
BRDF: Bidirectional Reflectance Distribution Function

θ : zenith

ϕ : azimuth

We only need two angles to define the direction





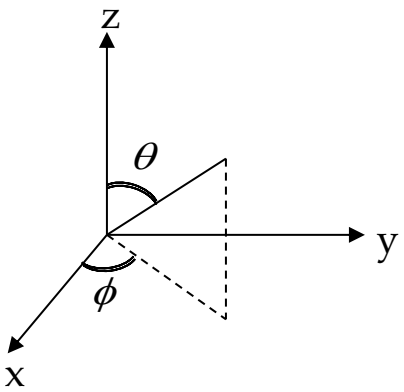
$E(\theta_i, \phi_i)$ Irradiance due to source in direction (θ_i, ϕ_i)

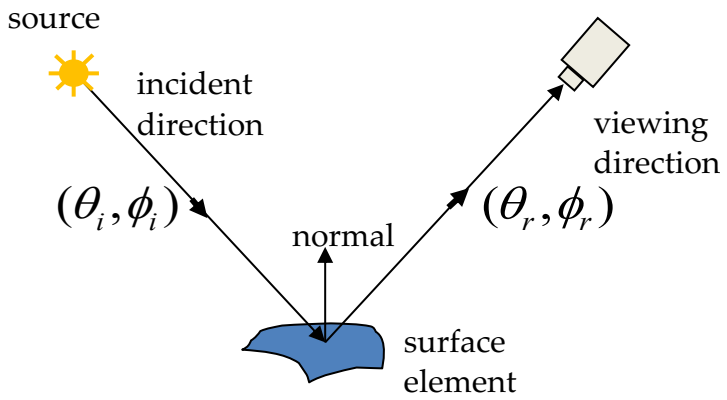
$B(\theta_r, \phi_r)$ Reflected energy of Surface in direction (θ_r, ϕ_r)

$$\text{BRDF}: \quad f(\theta_i, \phi_i; \theta_r, \phi_r) = \frac{B(\theta_r, \phi_r)}{E(\theta_i, \phi_i)}$$

(this is physics, from paper Nicodemus 1977)

BRDF Properties

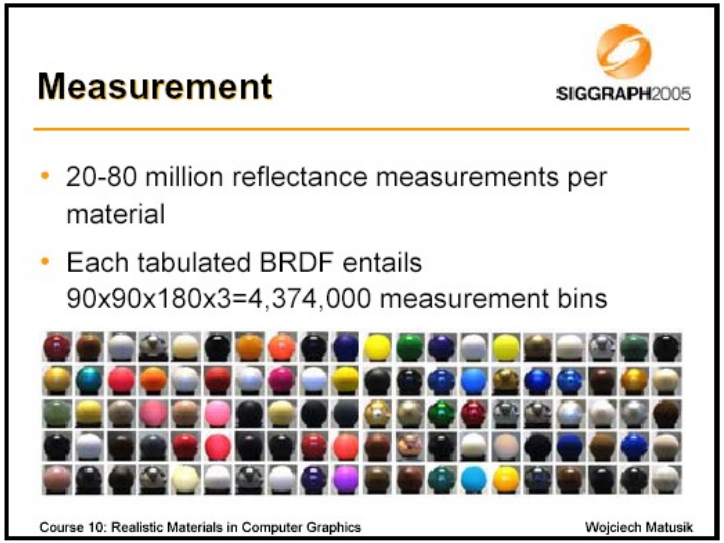




- BRDF is a 4D function
- Non-negative: $f(\theta_i, \phi_i, \theta_r, \phi_r) > 0$
- Helmholtz Reciprocity: $f(\theta_i, \phi_i, \theta_r, \phi_r) = f(\theta_r, \phi_r, \theta_i, \phi_i)$

BRDF databases

- MERL (Matusik et al.): 100 isotropic, 4 nonisotropic, dense



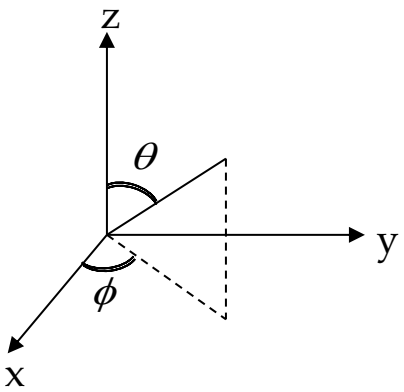
- CURET (Columbia-Utrect): 60 samples, more sparsely sampled, but also bidirectional texure functions (BTF)

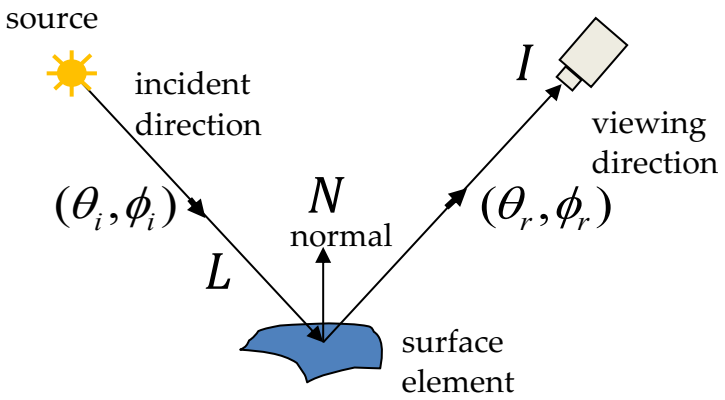
Sample albedos

Surface	Typical albedo
Fresh asphalt	0.04[6]
Open ocean	0.06[7]
Worn asphalt	0.12[6]
Conifer forest, summer	0.08,[8] 0.09 to 0.15[9]
Deciduous forest	0.15 to 0.18[9]
Bare soil	0.17[10]
Green grass	0.25[10]
Desert sand	0.40[11]
New concrete	0.55[10]
Ocean ice	0.50 to 0.70[10]
Fresh snow	0.80[10]
Aluminium	0.85[12][13]

[Wikipedia: Albedo]

BRDF Problems





- However, it is hard to solve N for the model $I = f(\theta_i, \phi_i, \theta_r, \phi_r)\, N \cdot L$
- because the surface normal is now in the BRDF function, which measures θ and ϕ; so, we need non-linear solvers
- We can do this more practically, such as using the Phong model

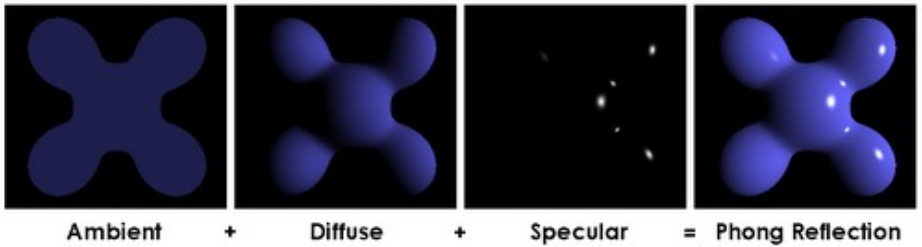
Phong Model

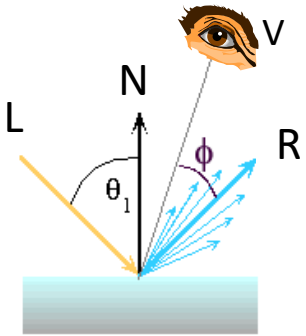
- The Phong model simplifies the BRDF math to

$$I = I_{ambient} + I_{diffuse} + I_{specular}$$

$$I_{diffuse} = k_d N \cdot L$$

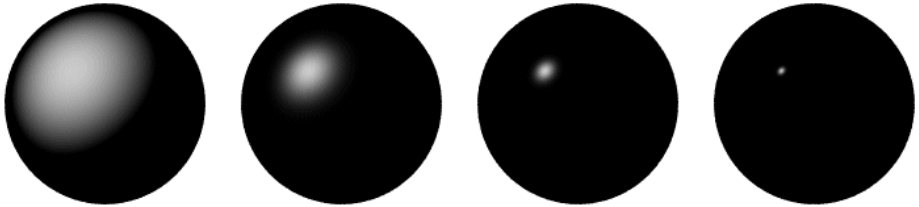
$$I_{specular} = k_s (V \cdot R)^{n_{shiny}}$$



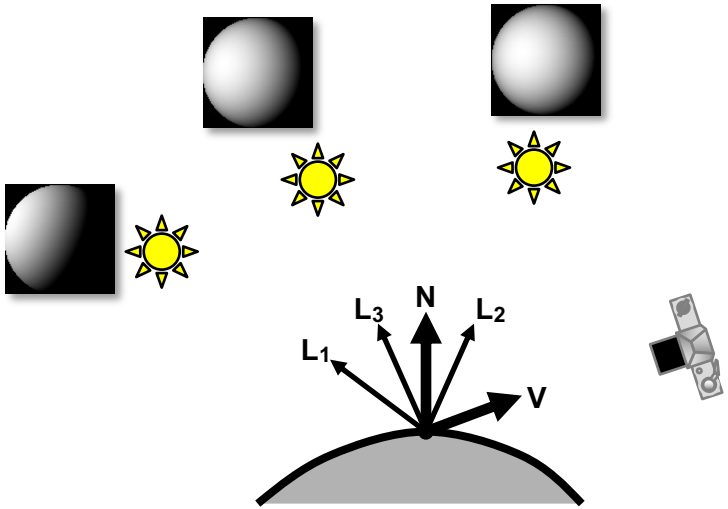


R is the perfect mirroring direction of L with respect to N

- k_s (specular reflection coefficient): how shiny the material is
- n_{shiny} (shininess constant): larger value means smaller and sharper highlight



Multiple Images: Photometric Stereo



We can use multiple light sources from different directions:

$$I_1 = k_d \mathbf{N} \cdot \mathbf{L_1}$$
$$I_2 = k_d \mathbf{N} \cdot \mathbf{L_2}$$
$$I_3 = k_d \mathbf{N} \cdot \mathbf{L_3}$$

Write this as a matrix equation:

$$\begin{bmatrix} I_1 & I_2 & I_3 \end{bmatrix} = k_d \mathbf{N}^T \begin{bmatrix} \mathbf{L_1} & \mathbf{L_2} & \mathbf{L_3} \end{bmatrix}$$

$[I_1 \ I_2 \ I_3]$ means the pixel intensity values at the same spot (that has surface normal N) across three photos

Solving the Equations

$$\underbrace{\begin{bmatrix} I_1 & I_2 & I_3 \end{bmatrix}}_{\mathbf{I}\ 1\times 3} = \underbrace{k_d \mathbf{N}^T}_{\mathbf{G}\ 1\times 3} \underbrace{\begin{bmatrix} \mathbf{L_1} & \mathbf{L_2} & \mathbf{L_3} \end{bmatrix}}_{\boldsymbol{L}\ 3\times 3}$$

Solving:

$$\mathbf{G} = \mathbf{I}\mathbf{L}^{-1}$$ (superscript -1 here means Inverse matrix)

Normalized the G, separate it into magnitude k_d and normal N

$$k_d = \|\mathbf{G}\|$$ (this is the L2 norm, i.e., Euclidean norm)

$$\mathbf{N} = \frac{1}{k_d}\mathbf{G}$$

Solving the Equations

$$\underbrace{\begin{bmatrix} I_1 & I_2 & I_3 \end{bmatrix}}_{\mathbf{I}} = \underbrace{k_d \mathbf{N}^T}_{\mathbf{G}} \underbrace{\begin{bmatrix} \mathbf{L_1} & \mathbf{L_2} & \mathbf{L_3} \end{bmatrix}}_{\boldsymbol{L}}$$

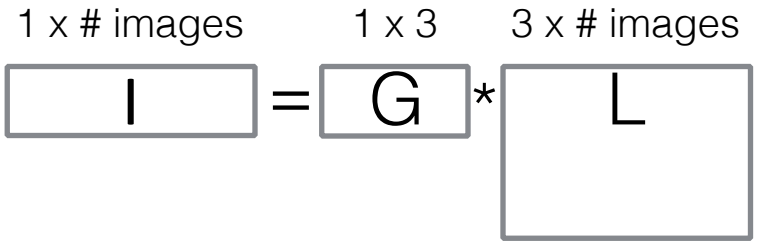
$$\underset{1 \times 3}{\mathbf{I}} \qquad \underset{1 \times 3}{\mathbf{G}} \qquad \underset{3 \times 3}{\boldsymbol{L}}$$

$$\mathbf{G} = \mathbf{I}\mathbf{L}^{-1}$$

- When is L nonsingular (invertible)?
  - Three light directions are linearly independent, or three light direction vectors cannot lie in a plane.
  - Otherwise, the matrix L will not be full rank and thus is not invertible
- What if we have more than one pixel location?
  - Stack them all into one big system (what does this mean for I and G ?)

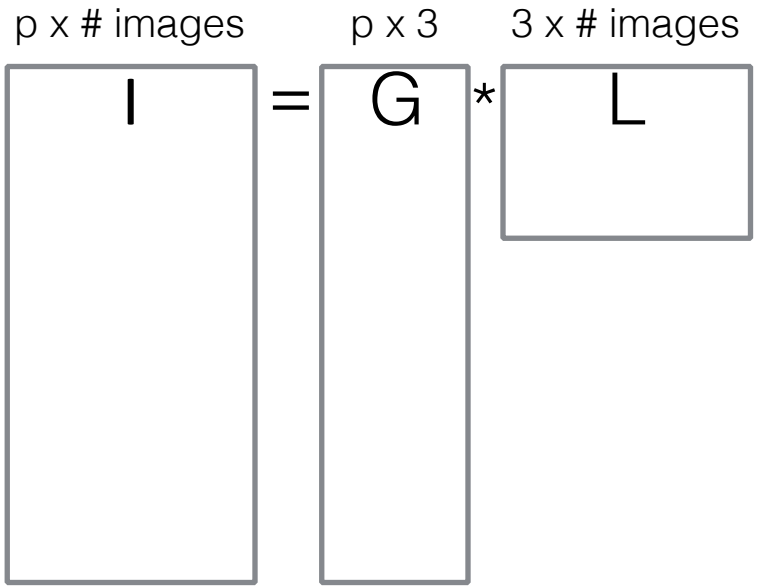
More than one pixel

Previously:



More than one pixel

Stack all pixels into one system:



Solve as before.

More than Three Lights

$$\underset{p \times n}{\begin{bmatrix} I_1 & \dots & I_n \end{bmatrix}} = \underset{p \times 3}{k_d \mathbf{N}^T} \underset{3 \times n}{\begin{bmatrix} \mathbf{L_1} & \dots & \mathbf{L_n} \end{bmatrix}}$$

- If we have more than three lights, we can use Ordinary Least Squares:

$$\mathbf{I} = \mathbf{GL}$$ (we cannot invert a non-square matrix L)

Find $\boldsymbol{G}$ s.t. minimizes $\|\boldsymbol{I} - \boldsymbol{GL}\|^2$

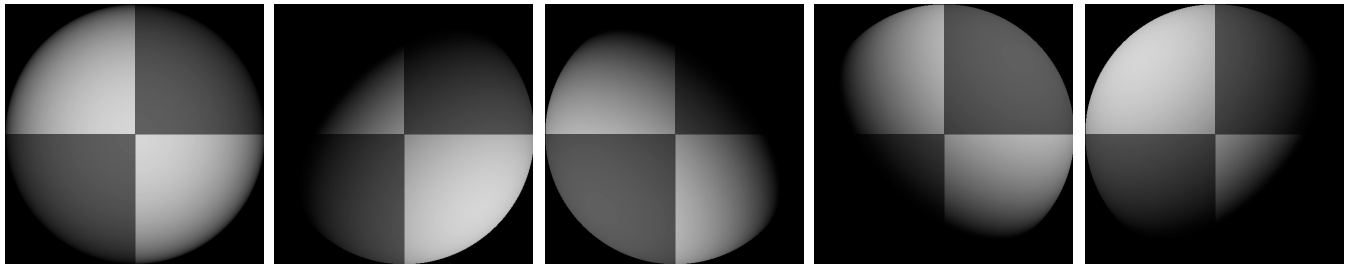
$$\mathbf{IL}^{\mathbf{T}} = \mathbf{GLL}^{\mathbf{T}}$$

$$\underset{p \times 3}{\mathbf{G}} = (\underset{p \times 3}{\mathbf{IL}^{\mathbf{T}}})(\underset{3 \times 3}{\mathbf{LL}^{\mathbf{T}}})^{-1}$$ (this is pseudo-inverse)

- Given G, solve for N and k_d as before.

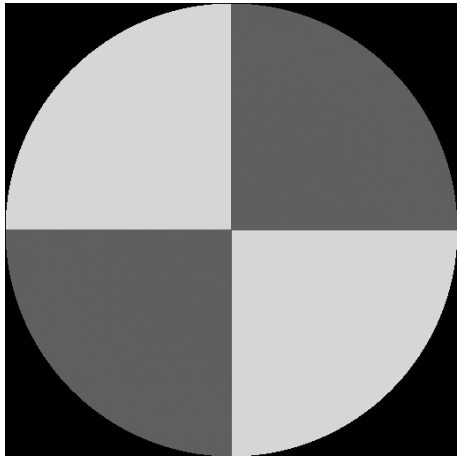
Example

Input views

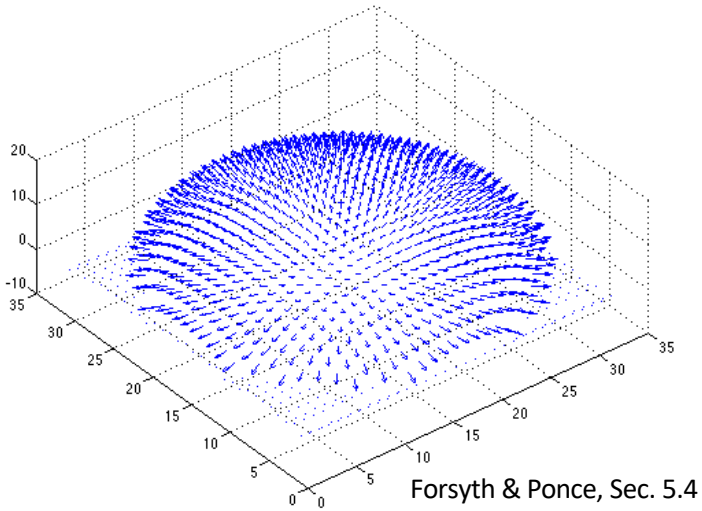




Recovered albedo



Recovered normal field



Forsyth & Ponce, Sec. 5.4

Depth from surface normals

- Solving the linear system per-pixel gives us an estimated surface normal for each pixel
- How can we compute depth from surface normals?
  - Surface normals are like the “derivative” of the true depth
  - We can do integrals on the surface normals to get the depth



Input photo | Estimated normals | Estimated normals (needle diagram)

[Woodham: Analysing images of curved surfaces. Artificial Intelligence, 1981]

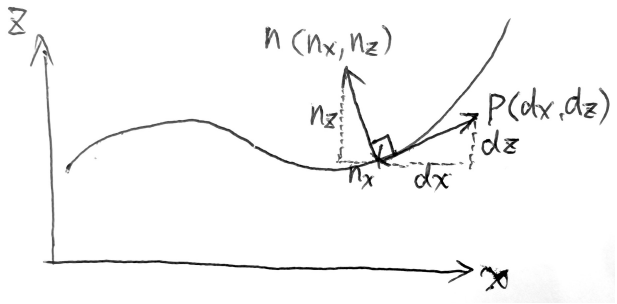
Normal Integration

- Calculate the partial derivatives from normal (n_x, n_y, n_z)

$$p(x,y) = \frac{\partial z}{\partial x} = \frac{-n_x}{n_z}$$

$$q(x,y) = \frac{\partial z}{\partial y} = \frac{-n_y}{n_z}$$

Why p and q look like this? → (hint: think about how to mathematically represent the fact that n and p are perpendicular)



- Then, integrate surface normals separately using the partial derivatives (Euler's method)

Along the x axis: $Z(x+1, y) = Z(x, y) + p(x, y)$
Along the y axis : $Z(x, y+1) = Z(x, y) + q(x, y)$

- Finally, verify if the partial derivatives can form a good surface using symmetry of second derivatives (i.e., equality of mixed partials)

Ideally, we should have $\frac{\partial}{\partial y}(\frac{\partial z}{\partial x}) = \frac{\partial}{\partial x}(\frac{\partial z}{\partial y})$ And you can check their differences

Depth Map from Normal Map

- However, errors accumulate. To get a better surface, we can formulate this as a linear system and solve for depths that *best agree with the surface normals.*

$$Z(x+1,y) - Z(x,y) = p(x,y)$$
$$Z(x,y+1) - Z(x,y) = q(x,y)$$

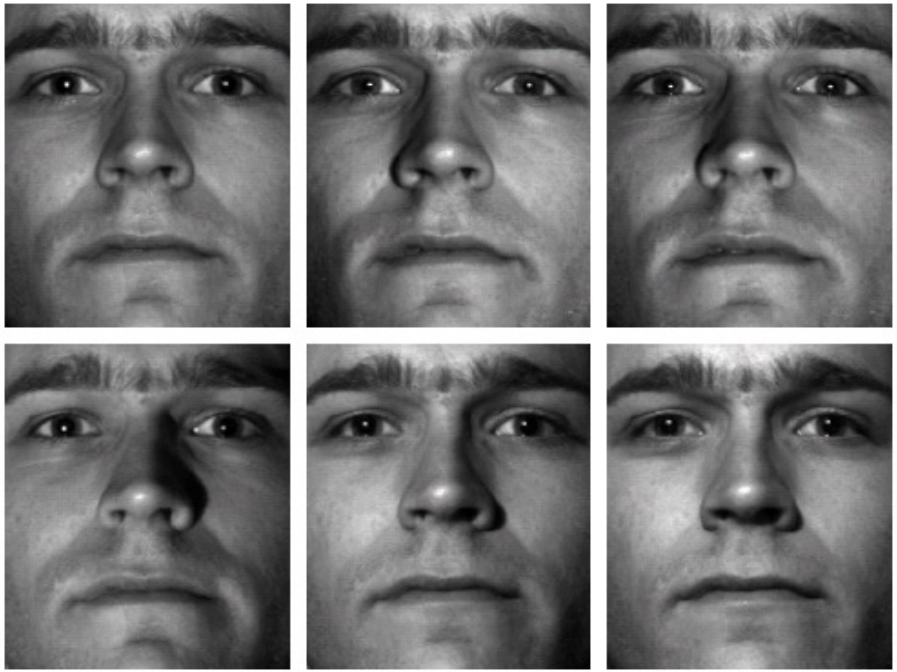
$$Z(1,0) - Z(0,0) = p(0,0)$$
$$Z(0,1) - Z(0,0) = q(0,0)$$
$$\vdots$$

$$Az = b$$

$$\begin{bmatrix} -1 & 0 & \cdots & 1 & \cdots \\ -1 & 1 & \cdots & 0 & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots \end{bmatrix} \cdot \begin{bmatrix} Z(0,0) \\ Z(0,1) \\ \vdots \\ Z(1,0) \\ \vdots \end{bmatrix} = \begin{bmatrix} p(0,0) \\ q(0,0) \\ \vdots \\ \vdots \\ \vdots \\ \vdots \end{bmatrix}$$

For more methods -- https://www.youtube.com/watch?v=tZ1mQRr9jiY&

Results

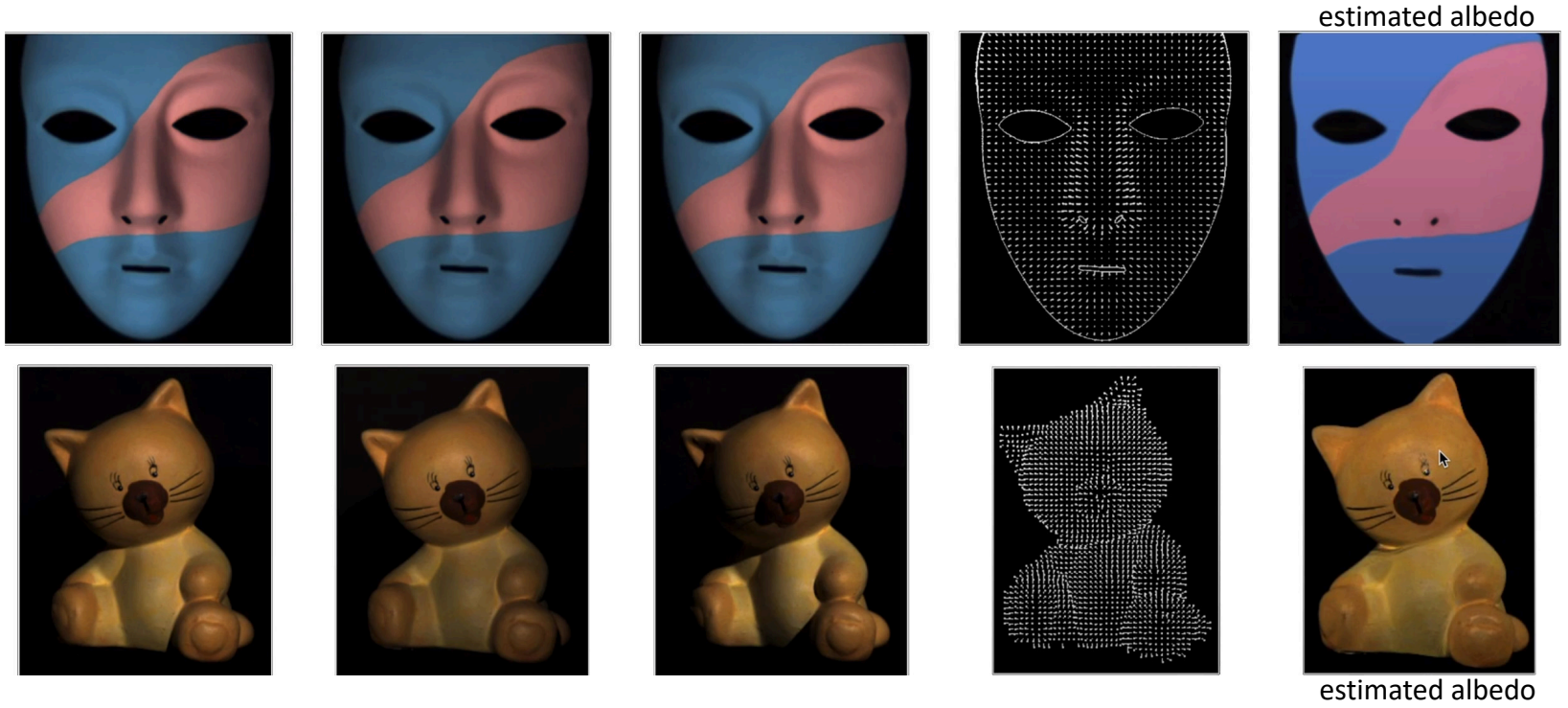




[from Athos Georghiades]

44

Results

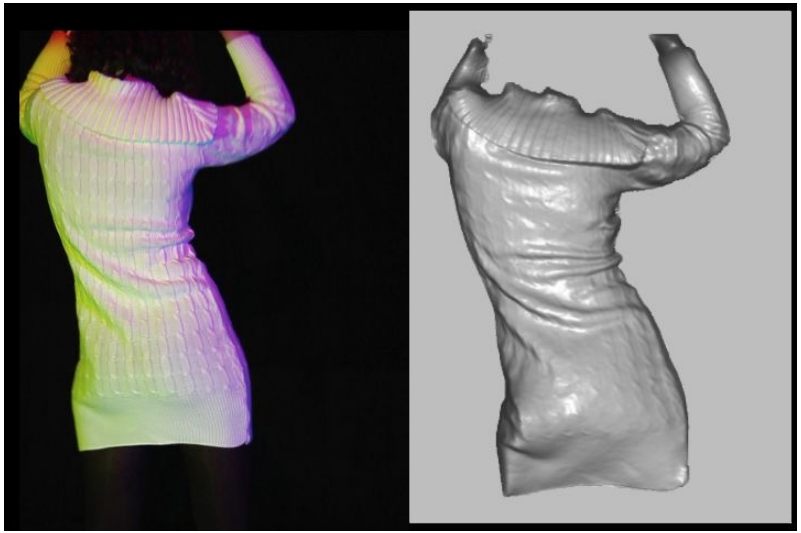


- For color images, apply the technique to each channel separately to obtain color albedo.
- However, deviations from Lambertian assumption and global illumination cause errors.

[slide credit: Andreas Geiger]

Extension

- Photometric Stereo from Colored Lighting



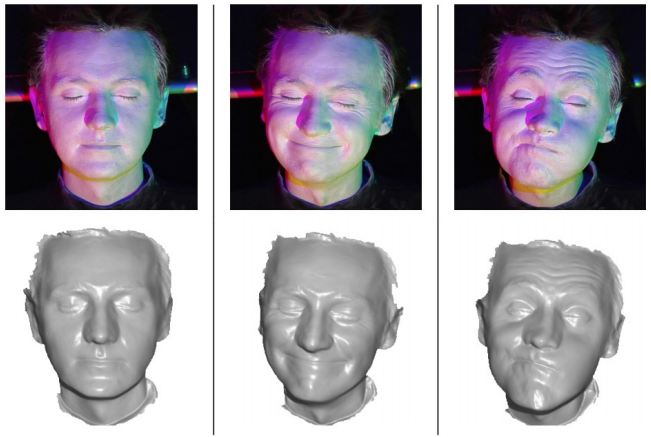


Fig. 2. **Applying the original algorithm to a face with white makeup.** Top: example input frames from video of an actor smiling and grimacing. Bottom: the resulting integrated surfaces.

Video Normals from Colored Lights
Gabriel J. Brostow, Carlos Hernández, George Vogiatzis, Björn Stenger, Roberto Cipolla
IEEE TPAMI, 2011.

Intrinsic Image Decomposition

elementwise
multiplication



Natural image

Reflectance

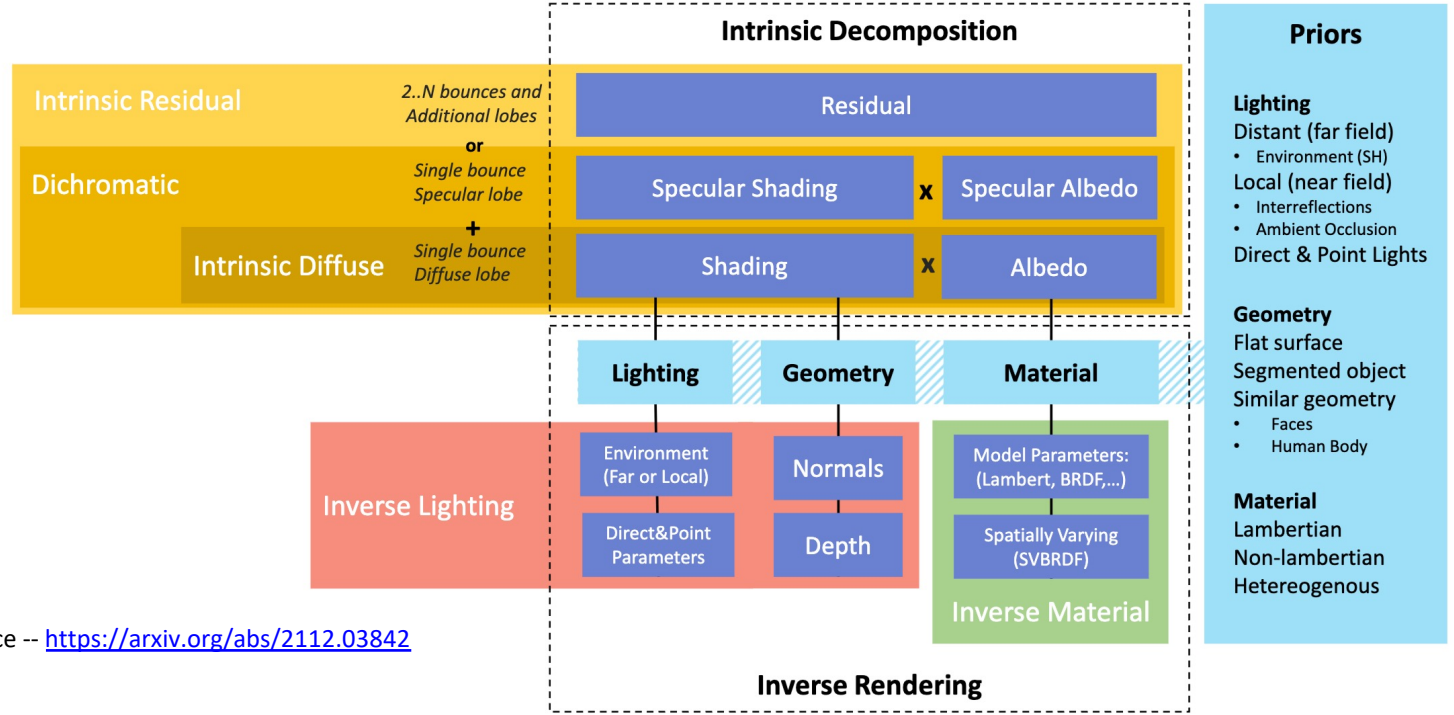
albedo
(material)

Shading

lighting and
geometry

Image from Liu et al., CVPR2020 -- Unsupervised Learning for Intrinsic Image Decomposition from a Single Image

Intrinsic Image Decomposition



source -- https://arxiv.org/abs/2112.03842

Fig. 5 Taxonomy of single image inverse appearance reconstruction methods: intrinsic decomposition and inverse rendering. The intrinsic diffuse model assumes Lambertian materials and jointly couples light and geometry interaction through a shading image. The intrinsic residual captures a residual image of the sum of the other non-diffuse effects. In between, several methods in the literature have used the dichromatic reflection model, where only a colored specular reflection is taken into account. Inverse rendering methods aim at recovering the full parameterized scene (camera, lighting, geometry, and material) to synthesize novel views or to enable predictably edits. To simplify the problem, many methods impose priors over the scene elements, *e.g.*: distant lighting through spherical harmonics illumination, or known proxy geometries (faces, human bodies,

Total Relighting



[Pandey et al, SIGGRAPH'21, YouTube]

Image processing operations

- Point-wise operations
  - histogram operation
- Neighborhood operations
  - Spatial domain (image convolution)
  - Frequency domain (Fourier transform)
  - Binary image analysis
- Geometric operations
  - Rotation, translation, affine…

Histograms

How do we ‘summarize’ images?

many images of this part are from https://rvdboomgaard.github.io/ComputerVision_LectureNotes/index.html

Image Histograms

1D / Univariate Histogram



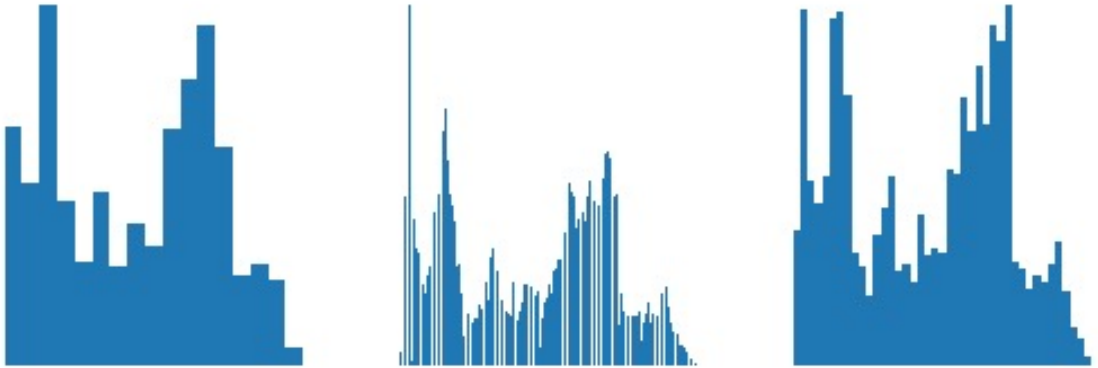
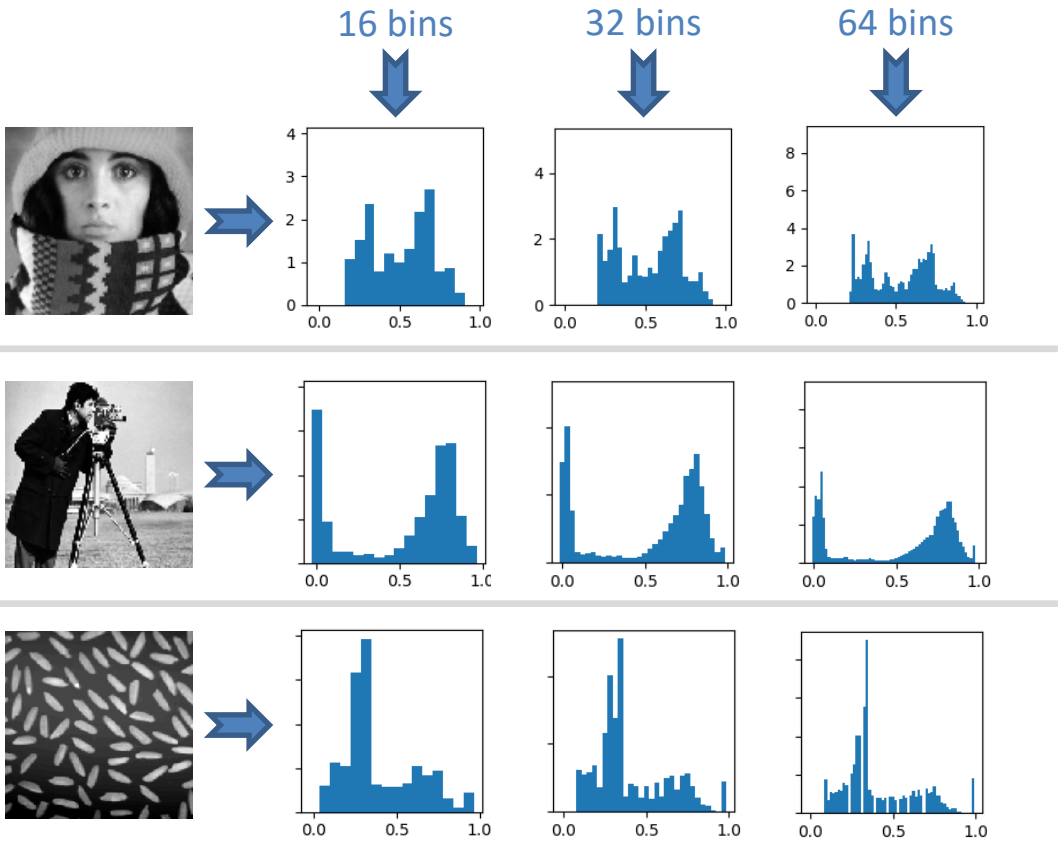


Image Histograms



Too many (and too small) bins?

→ Many bins will be empty

→'High-frequency' info

Too few (and too big) bins?

→ Will not encode 'well enough' the distribution of values

Image Histograms

Histogram → $h_f(u)$ counts pixels with value $f(x) = u$ — ***PDF*** – Probability-density function, normalized such that: $\sum h_f(u) = 1$

Cumulative Histogram → $H_f(u)$ counts pixels with value $f(x) \leq u$ — ***Cumulative distribution*** function



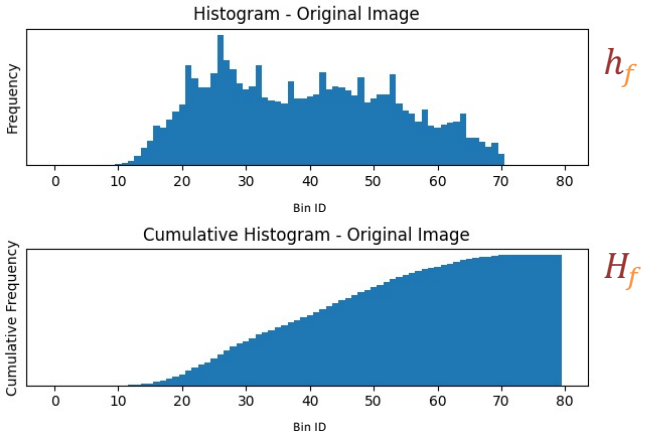
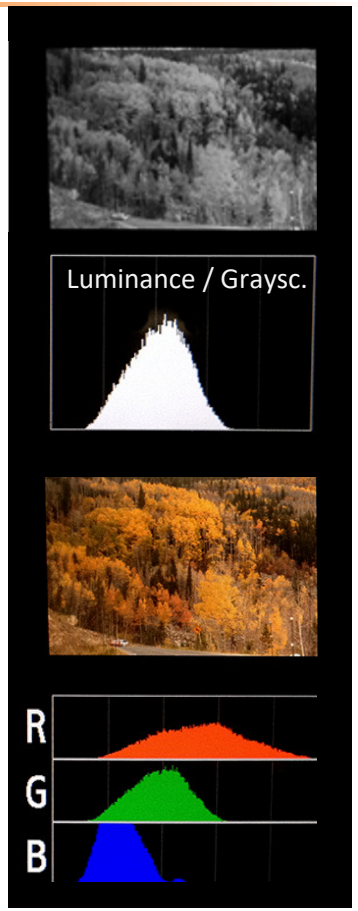


Image Histograms

3D / Multivariate Histogram

Grayscale images → ***1D Histogram***

(each pixel → Scalar value) (Univariate)



What about ***Color*** images?

(each pixel → Triplet of BGR value)

$$\boldsymbol{f}(\boldsymbol{x}) = \begin{bmatrix} f_1(\boldsymbol{x}) \\ f_2(\boldsymbol{x}) \\ f_3(\boldsymbol{x}) \end{bmatrix} = \begin{bmatrix} b(\boldsymbol{x}) \\ g(\boldsymbol{x}) \\ r(\boldsymbol{x}) \end{bmatrix}$$

Ignores color **Correlations**!

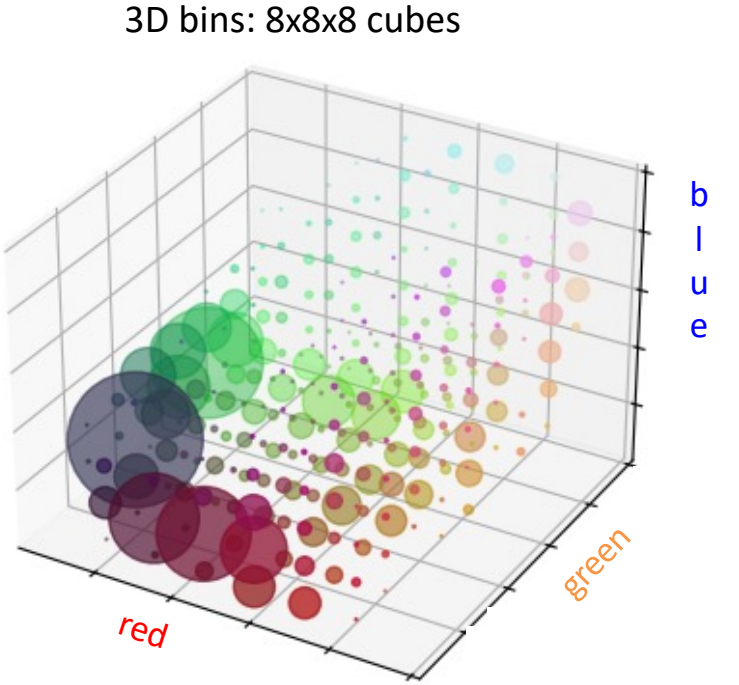
Trivial solution: ***3x separate*** 1D histograms per channel

Image Histograms

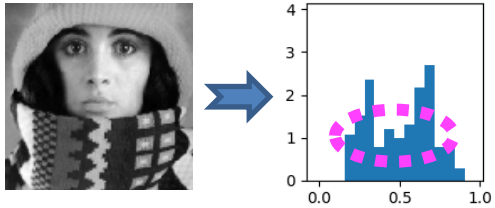
3D / Multivariate Histogram



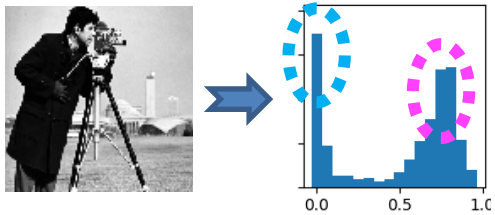




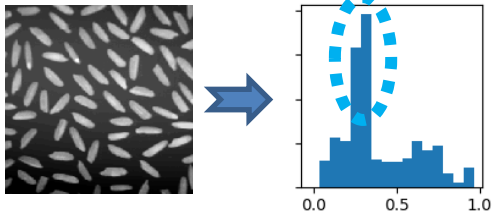
Problems about image histograms



Only *mid-range* gray
No extremes



Spikes for *bright* and
very-dark areas



Mostly *dark-ish* areas

Not using full range of luminance



Image might be … :

Missing *large (bright)* values? → … too *dark*

Missing *low (dark)* values? → … too *bright*

Changing illumination conditions
for same scene → very different histograms!





Point Operators

We can use “point operators” to manipulate histograms to mitigate these problems.

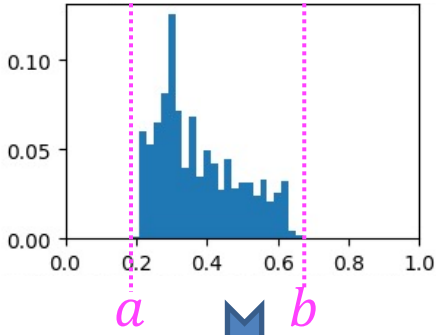
A point operator is an image processing function that transforms each pixel’s original value independently (and not based on its neighbors).

Histogram Contrast Stretching

X axis: luminance values (bins)
Y axis: frequency for each value

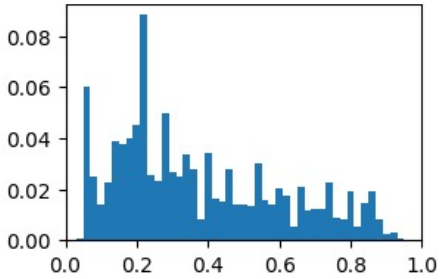
Input:
Low Contrast





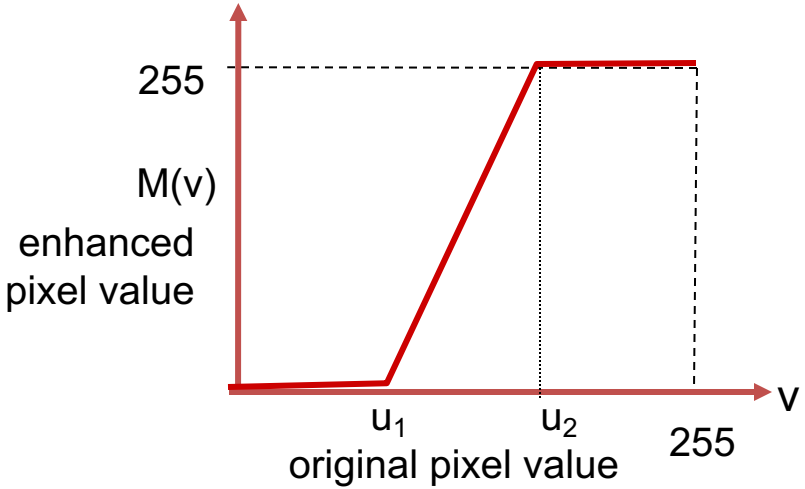
Output:
Contrast Stretched

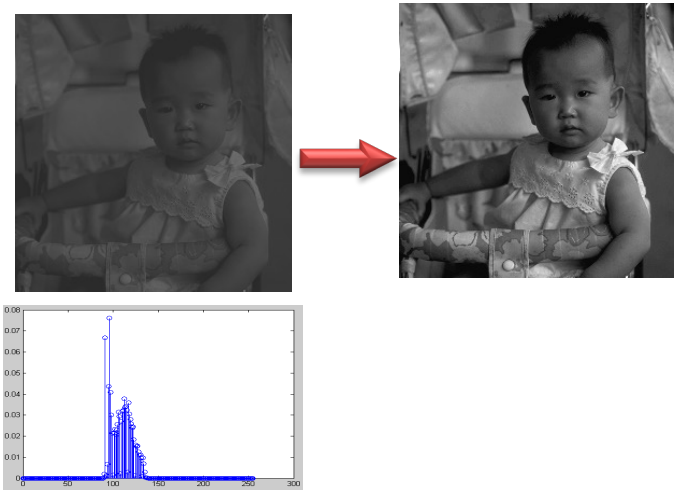




Contrast Enhancement

- If most of the gray-levels in the image are in $[u_1\ u_2]$, the following mapping increases the image contrast.
- The values u_1 and u_2 can be found by using the image's accumulated histogram.





Gamma Correction

Non-linear scaling (power-law transformation):

Output gray level $s(r) = cr^{\gamma}$

Normalization constant

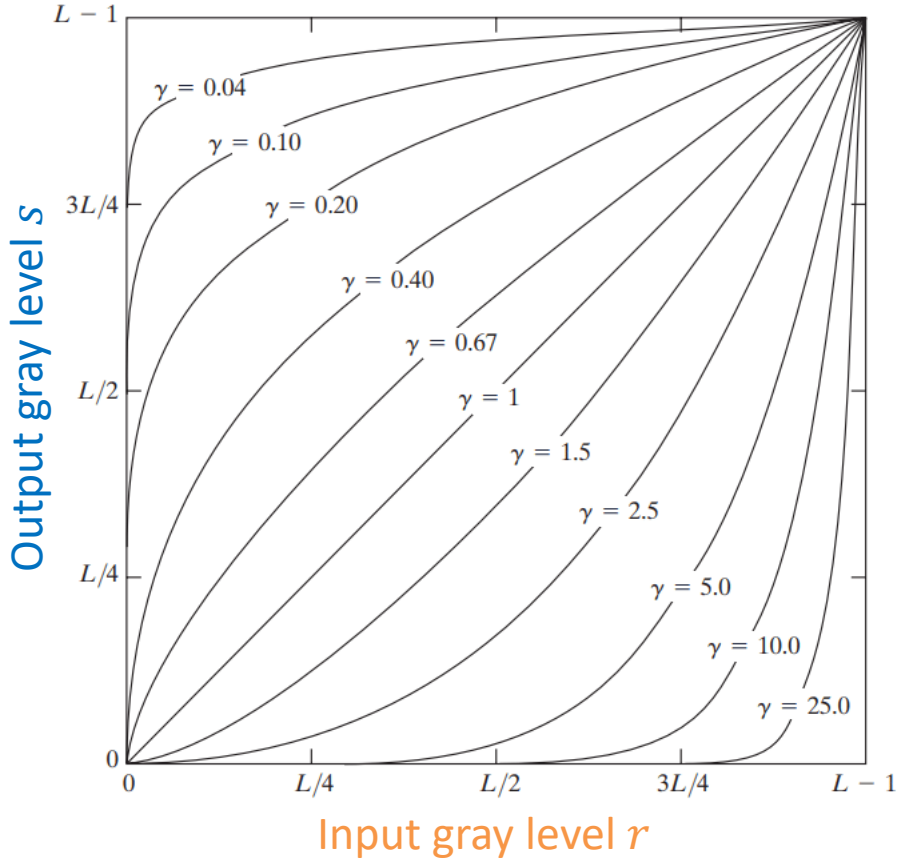
Input gray level

- Note: $s(r)$ is linear for $\gamma = 1$.



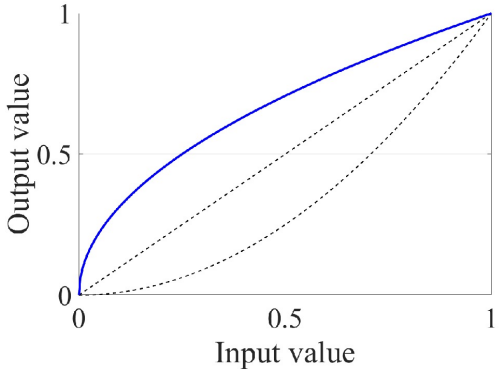
gamma=0.5 gamma=1.0 gamma=1.8 gamma=2.0 gamma=2.5

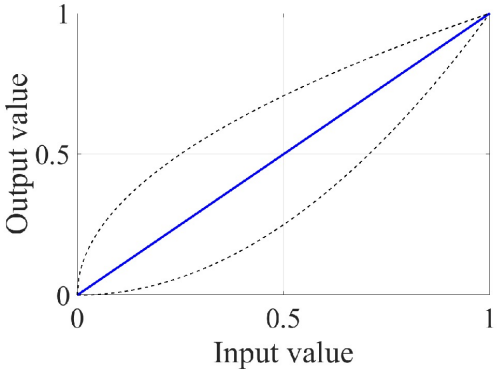
[Artlets.com]

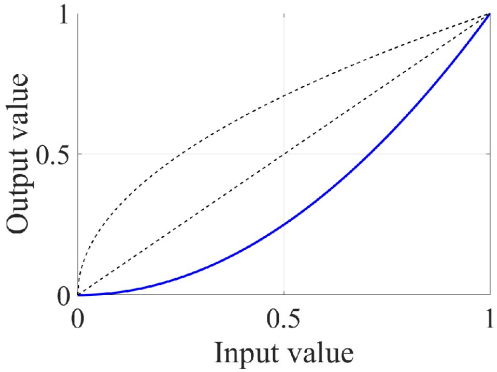


[Yin at al., 2022]

Gamma Correction













[Ngo and Kang, Electronics, 2021]

Histogram Equalization (idealized)

Produce image $G = f(I)$ with ***'constant' (flat) histogram*** h_G

All pixel values should be 'equally' probable
(as 'equal' as possible for finite number of pixels)

Example, we assume:

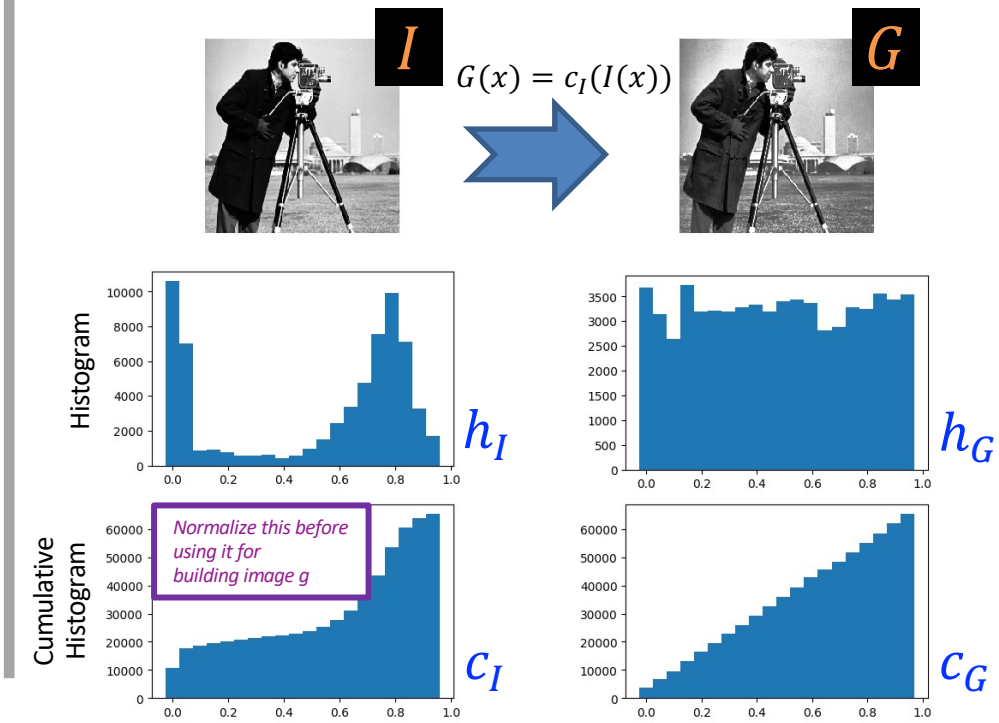
- image size 100*100
- image is grey scale with range 0 to 255
- 256 bins for the histogram
- We have 10 pixels with value 0; 100 pixels with value 1
- for all pixel location x in I, we have pixel value $I(x)$

Step:

- Step 1 -- compute h_I and c_I; in the example, $h(0) = 10$ and $h(1) = 100$, so $c(1) = 110$
- Step 2 -- normalize c_I so that its output is in the desired range, such as 0 to 255
- Step 3 -- construct image $G(x) = c_I(I(x))$, where x is the pixel location in image I and G; in other words, we use c_I as the point operation function

Why this works?

- Intuitively, we replace the pixel values with its corresponding percentile, which means the histogram of $G(x)$ is flat



Additional Resources

- Photometric Stereo
  - Shree Nayar: YouTube
  - Andreas Geiger: YouTube
- Specular and Diffuse Reflection
  - Steve Seitz: YouTube / YouTube

Disclaimer

Many of the slides used here are obtained from online resources (including many open lecture materials) without appropriate acknowledgement. They are used here for the sole purpose of classroom teaching. All the credit and all the copyrights belong to the original authors. You should not copy it, redistribute it, put it online, or use it for any other purposes than for this course.